



Semester Project: FALL 2025

Data Visualization for MOT Database

COMP 410 Parallel and Distributed Computing

Due on 1st December 2025, 11: 59 PM

Late Submission Policy (15% deduction for one day late, 30% for 2 days late, 60% for 3 days late)

You are allowed to Do this project in a group of three or pairs.

You have to present your project in the class on the shared schedule.

Work should be evenly distributed.

Introduction

The UK's Department for Transport (formerly Ministry Of Transport, or MOT) makes available an anonymized dataset containing the results of every MOT test performed on a road vehicle each year [1]. In addition to the date of the test and the result, the dataset contains the make and model of every vehicle tested, the date on which it was first registered and mileage at the date of the test. In this assessment, you are going to create a desktop Python application capable of browsing the data contained within this dataset and performing some basic statistical analysis.

Acquiring & Understanding the Dataset

The dataset is available [here](#), and appears to be published annually. You will be using data that has been published in 2021, and this is the year's data upon which you will be working. You will need to download a copy of the data from the site, using the “[MOT Testing Data Results \(2021\)](#)” link. You will be working only with the test results dataset, and not the failure item dataset, which contains data on what exactly caused vehicles to fail their MOT test. Take care to ensure that you download the correct file.

The .zip downloaded from the Department for Transport website contains a number of large Comma Separated Value (CSV) files containing the MOT test data split by reporting period for the year. A CSV file is a plain text file containing tabular data similar to that in a spreadsheet or database table. Each line in a CSV file contains the data for one particular row of a table (i.e. one test). The first row of each CSV file, as is common, is a header row that gives the names of the columns in the file, to aid in understanding the data structure. The columns contained within the MOT test data CSV files are as follows, in the specified order:

```
test_id,  
vehicle_id,  
test_date,
```

test_class_id,
test_type,
test_result,
test_mileage,
postcode_area,
make,
model,
colour,
fuel_type,
cylinder_capacity,
first_use_date

The *test_id* column contains a unique identifier for each test.

The *vehicle_id* column contains an identifier that uniquely identifies vehicles under test while avoiding disclosing registration numbers and maintaining anonymity.

The *test_date* column contains the dates tests were conducted, in ISO 8601 (yyyy-mm-dd)

format. The *test_class_id* column specifies the class of vehicle tests conducted. For more details see [2].

The *test_type* column specifies the types of test conducted (e.g. NT for normal test, RT for retest, PL/PV for retests conducted after fixing a minor identified problem)

The *test_result* column contains the results of tests conducted (e.g. P for pass, F for Fail, PRS for pass with a rectification to be carried out at the testing station)

The *test_mileage* column contains the mileage of vehicles tested, at the time of the tests.

The *postcode_area* column contains the alphabetical characters of the first part of the testing centre's postcode, which serves as an indicator of the town/city/area in which the test was conducted.

The *make*, *model* and *colour* columns are all self-explanatory. **Do note, however, that some of these values contain commas, and have thus been surrounded by quotation marks in the files. Some values contain quotation marks, and these have been escaped with a backslash.**

The *fuel_type* column indicates the type of fuel used by vehicles. Notable values include PE (Petrol), DI (Diesel), EL (Electric) and ST (Steam).



The *cylinder_capacity* column contains the capacity of vehicles' engine cylinders, commonly measured in cc. Vehicles' engine capacities are calculated by multiplying by the number of cylinders.

The *first_use_date* column contains the date on which vehicles were first registered for use in ISO8601 format (yyyy-mm-dd), allowing their age at date of test to be calculated in combination with the *test_date* column.

After the first (header) row, each test's data is contained on a line of the CSV file. The values for each column are separated by commas which makes it fairly easy to parse the data programmatically. An example line from one of the files is depicted below:

```
1449915597,1108091828,2020-12-31,4,NT,PRS,128135,E,BMW,3  SERIES,SILVER,PE,2996,2007-06-29
```

To understand the data in the example above, correlate each of the values (separated by commas) with the associated column by consulting the column names and ordering on the previous page, or on the header line of the CSV file.

Your Task

You are going to create a Python application capable of browsing the MOT test data you have downloaded and performing some basic statistical analysis. Because the data is fairly sizeable (approx 3GB), you will load data in the data structure of your own choice (Linked list, queue, tree, Arrays, Pandas Dataframe, Trie, Dictionary etc.) that allows the data to be searched efficiently. For saving time you can pick fifty thousand entries from each file.

You will need to submit a Zip file (.zip) containing the following:

1. The source code of a Python program that imports all of the CSV data you downloaded into your data Structure. Use relative paths in your code, also clean your data The database itself will be too large for submission, so be sure not to include that in your exported work.
2. The source code of a Python application that allows users to query the created database to view specific test results and identify trends.
3. Some screenshots depicting the results of specified queries in your application.
4. Project Report how you have used your Course knowledge to divide and optimize the tasks on different Machines.
 - a) MPI
 - b) Decomposition & Mapping Techniques
 - c) What model/ Algorithm you have used
 - d) Optimization you have performed.
 - e) Bonus: if you were able to perform cluster computing

You can find more details on what exactly you need to do for each of these tasks in the following sections.

You will need to create a program capable of importing the data contained within the downloaded CSV files into your data structure using CSV. The program does not need any kind of user interface, nor does it need to accept any input other than the CSV files themselves. I would recommend that you create the program such that it searches its current working directory for all CSV files, and loads data. You can use below relation to create efficient data structure.

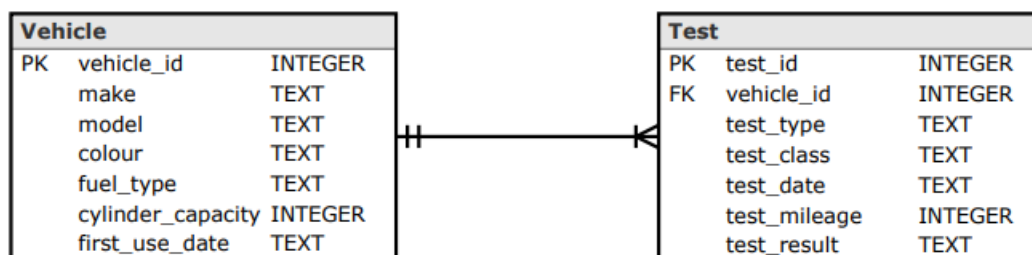


Figure 1: Entity-Relation Model of Required Database Table Structure



In addition to the tables depicted in Fig. 1, above, you may create additional indices to improve the performance of your application.

Your program will need to INSERT first 1,000,000 instances from each file of the data in the downloaded CSV files into the tables created by your program. This may take a considerable length of time, so your program should output a progress update every thousand rows, and at the start/end of each file processed.

Your data import program will be assessed on the following criteria:

- Correctness of the generated database (20%)
- Quality of the code (10%)

Be careful to handle fields that are enclosed in quotation marks, or that contain escaped commas, correctly. If you make a mistake here, some rows will have values in the wrong columns.

You may wish to add a flag into your data insert program that makes it stop after the first 100,000 rows in each file when developing and testing, so that you can iron out issues without waiting for the entire file to be processed.

Implementation (70%)

You will create a Python application having CLI interface (GUI will give you 2 ABS bonus marks) that allows users to browse the data contained within the database created in the previous section, and to perform some basic statistical analysis on said data. Your application will be assessed on the following criteria.

- Implementation of basic search and display functionality (10%)
- Optimization of each program (15%)
- Implementation of analysis and reporting functionality (25%)
- Better user interface (5%)
- Quality of the code (15%)

Here's a simplified outline of the process:

- User initiates a search.
- Query is divided into sub-queries.
- Each VM/ Node (MPI node) receives a sub-query for processing.
- Local processing on each VM/node using database.
- Results of sub-queries are sent back to the coordinator using MPI.
- Coordinator aggregates results into a comprehensive result set.

- Combined result set is displayed to the user on the coordinator machine.

Keep in mind that the efficiency and performance of this setup depend on the nature of the queries, the data distribution, and the overall system configuration. Testing and optimizing the MPI implementation will be crucial for achieving good performance.

Basic Search & Display Functionality

The basic search and display functionality should allow the user to search the database for all tests performed on a particular make of vehicle, a particular model of vehicle, vehicles first used in a particular year, vehicles by mileage at the date of test was within a specified range, or any combination thereof. On performing a search, users should be able to view all matching tests with all the aforementioned columns, plus the test result, should display all of the data held on that particular test.

Analysis & Reporting Functionality

The analysis and reporting functionality should allow the user to view graphs depicting the pass rates for vehicles of a specified make and model by age or by mileage. The graphs should be produced using Matplotlib or other graphing library, for which there is a wide variety of documentation and code samples available. You have discretion over the style of the graphs, and the grouping of age and mileage ranges in their production, but I would suggest creating a line chart grouping age by year and grouping mileage by ten thousand mile intervals initially.

You don't need to implement every feature in the assignment specification, you will be given credit for what you do complete, even if some sections are only partially completed.

You are required to use your OOP and data structure knowledge. Not sure but you can use pickle or dump for making things easier for second part of code.

Screenshots

To demonstrate your program working, you will include screenshots of your application performing various tasks in your submission. You do not need to collate the screenshots into a document or report. Instead, you should create a screenshots folder and save the screenshots, with descriptive filenames, into this folder. The screenshots will be used to verify the functionality of your program in conjunction with a review of the program's code, and features missing their associated screenshots will be presumed non-functional. You only need to include screenshots of the functionality you have functioning correctly; partially implemented features will be assessed when reviewing the code.

Basic Search & Display Functionality Screenshots

Your screenshots should depict the list of search results for each of the following criteria. For each, the first test result should be selected with all associated details visible in the program.

- BMW 3 series vehicles
- Ford Focus vehicles first used in 2014
- All Volkswagen vehicles with a mileage of between 100k and 120k miles at date of test
- All vehicles first used in 2015 and under 2k miles at date of test
- All vehicles under -1 miles at date of test.
- All vehicles first used in 2055

Analysis & Reporting Functionality Screenshots

Your screenshots should depict the your program's output for each of the following reports.

- Pass rate of Peugeot 207 vehicles by age
- Pass rate of Kia Rio vehicles by age
- Pass rate of Ferrari 458 vehicles by mileage

These queries should display the Chart output.

3 ABS bonus Points: if you are able to use cluster computing or use the computing power of two/three computers combined for optimization of above tasks.